Dossier de projet professionnel

Guangquan Ye

Concepteur Développeur d'Applications

1. Introduction 4
1.1 Présentation du contexte du projet 4
1.2 Problématique 4
1.3 Objectifs de l'application 4
2. Analyser les besoins et maquetter une application 4
2.1 Présentation du contexte 4
Description du client 5
2.2 Recueil des besoins 5
2.2.1 Méthode de recueil 5
2.2.2 Besoins fonctionnels 6
2.2.3 Besoins non fonctionnels 6
2.3 Maquettage 8
Outil utilisé 8
Maquettes écran par écran 8
exemple de maquette partie d'administrateur : 9
2.4 Parcours Utilisateur 9
2.5 Dossier de conception 10
3. Définir l'architecture logicielle 14
3.1 Choix de l'architecture 14
3.2 Architecture multicouche 15
3.4 Sécurité 17
Authentification 17
Le projet utilise le système d'authentification de Supabase : 17
● Le front réalise la connexion et obtient un JWT 17
● Le back-end vérifie le token sur les routes protégées 17
Autorisation 17
L'accès à l'administration est contrôlé par un rôle (ex. admin) 18
● Certaines routes REST sont accessibles uniquement si le token est valide et que le rôle requis est présent 18
● Le front protège également les routes /admin côté interface 18
Gestion des mots de passe 18
Le hashage et la gestion des mots de passe sont délégués à Supabase Auth (GoTrue). L'API Go ne manipule pas directement les mots de passe 18
Protection contre les vulnérabilités 18
Injection SQL
Utilisation de GORM et de requêtes paramétrées évitant la concaténation de requêtes SQL 18
XSS 18
● Validation et contrôle des contenus affichés 18
CSRF 18
● Utilisation d'un token Bearer dans le header Authorization 18
● Pas de cookies de session classiques côté API 18
● Configuration CORS limitant les origines autorisées 18
3.5 Éco-conception 18
4. Concevoir et mettre en place une base de données relationnelle 19

4.1 Analyse des données..... 19
Entités et rôle fonctionnel..... 19
Relations et cardinalités..... 20
Lien avec les besoins fonctionnels..... 20
4.2 Modèle Conceptuel de Données (MCD)..... 20
Points principaux..... 20
4.3 Modèle Logique de Données (MLD)..... 21
Tables et relations principales..... 21
Clés et intégrité..... 22
Gestion des utilisateurs..... 22
4.4 Modèle Physique de Données (MPD)..... 22
4.5 Règles de nommage..... 24
4.6 Intégrité et sécurité..... 24
4.7 Jeu d'essai..... 25
4.8 Documentation technique..... 26
5. Développer des composants d'accès aux données SQL et NoSQL..... 27
5.1 Technologies utilisées..... 27
5.2 Développement des composants..... 27
5.3 Gestion des exceptions..... 28
5.4 Validation des entrées..... 28
5.5 Sécurité et middlewares..... 29
5.6 Tests..... 29
Conclusion..... 30

1. Introduction

1.1 Présentation du contexte du projet

Le projet consiste à concevoir et développer un site vitrine artistique destiné à un artiste indépendant souhaitant exposer ses œuvres au sein d'une galerie en ligne personnelle.

Avant la mise en place de cette solution, les créations de l'artiste étaient diffusées sur différentes plateformes externes (réseaux sociaux, marketplaces), entraînant une dispersion des contenus, un manque de cohérence visuelle et une dépendance à des services tiers.

La création d'un site dédié vise à centraliser l'ensemble des œuvres dans un environnement maîtrisé, structuré et accessible sur tous les supports (ordinateur, tablette, mobile). Le site intégrera également un espace administrateur permettant à l'artiste de gérer ses contenus de manière autonome.

1.2 Problématique

L'artiste ne disposait pas d'un espace numérique centralisé lui permettant de présenter son travail de manière professionnelle et organisée.

1.3 Objectifs de l'application

L'application a pour objectifs de :

- Présenter les œuvres sous forme de galerie organisée par catégories et sous-catégories.
- Permettre leur gestion via un espace administrateur sécurisé.
- Rediriger vers une plateforme externe pour l'achat des œuvres dans une première version.
- Préparer une évolution vers une solution e-commerce intégrée.

2. Analyser les besoins et maquetter une application

2.1 Présentation du contexte

Description du client

Le client est un particulier exerçant une activité artistique en parallèle de son activité professionnelle principale. Il réalise des œuvres originales qu'il souhaite exposer et valoriser à travers une plateforme numérique dédiée.

L'objectif n'est pas seulement commercial, mais également artistique et identitaire : disposer d'un espace personnel permettant de présenter son univers visuel de manière professionnelle et structurée.

Contexte métier

Bien que l'activité artistique ne constitue pas son activité principale, le client souhaite développer sa visibilité et structurer la présentation de ses créations. Jusqu'à présent, la diffusion des œuvres s'effectuait via des plateformes tierces, sans maîtrise complète de la présentation, de l'organisation et du référencement.

Le besoin exprimé est donc la mise en place d'un outil numérique permettant :

- Une présentation centralisée des œuvres
- Une organisation par catégories et sous-catégories
- Une gestion autonome des contenus
- Une préparation à une éventuelle évolution vers la vente directe

2.2 Recueil des besoins

2.2.1 Méthode de recueil

Les besoins ont été recueillis à travers :

- Des échanges directs avec le client (entretiens informels et discussions structurées)
- L'analyse des attentes fonctionnelles exprimées
- La rédaction d'un cahier des charges formalisé à la suite de ces échanges

Le cahier des charges a donc été élaboré conjointement afin de structurer les besoins identifiés, de préciser les fonctionnalités attendues et de définir les contraintes techniques et graphiques du projet.

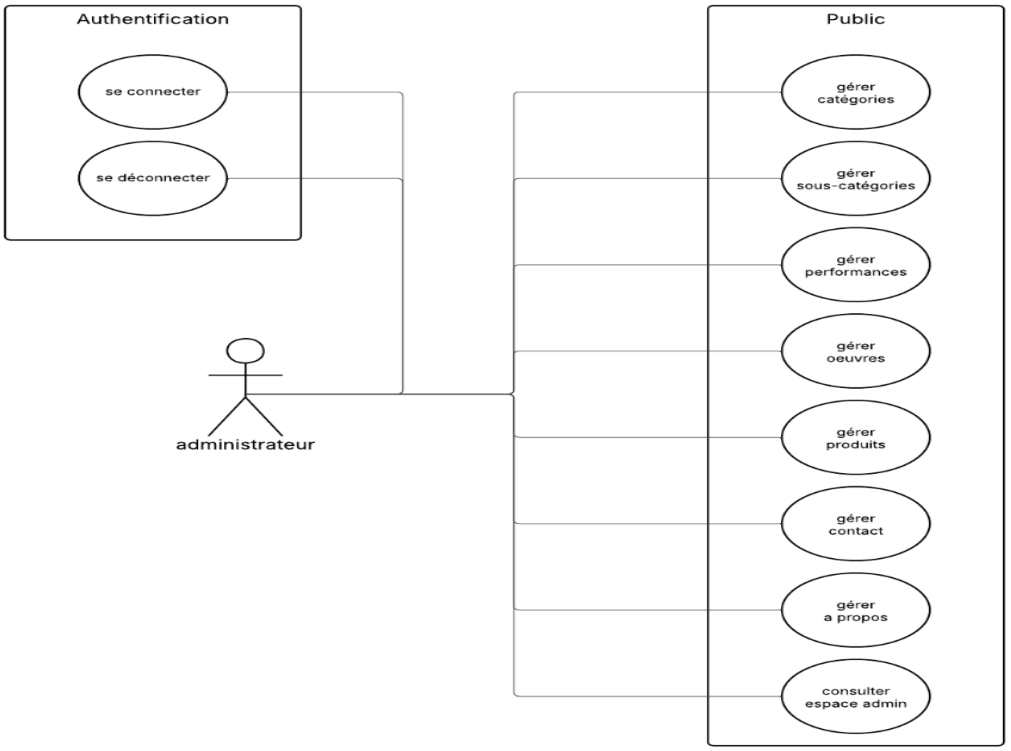
2.2.2 Besoins fonctionnels

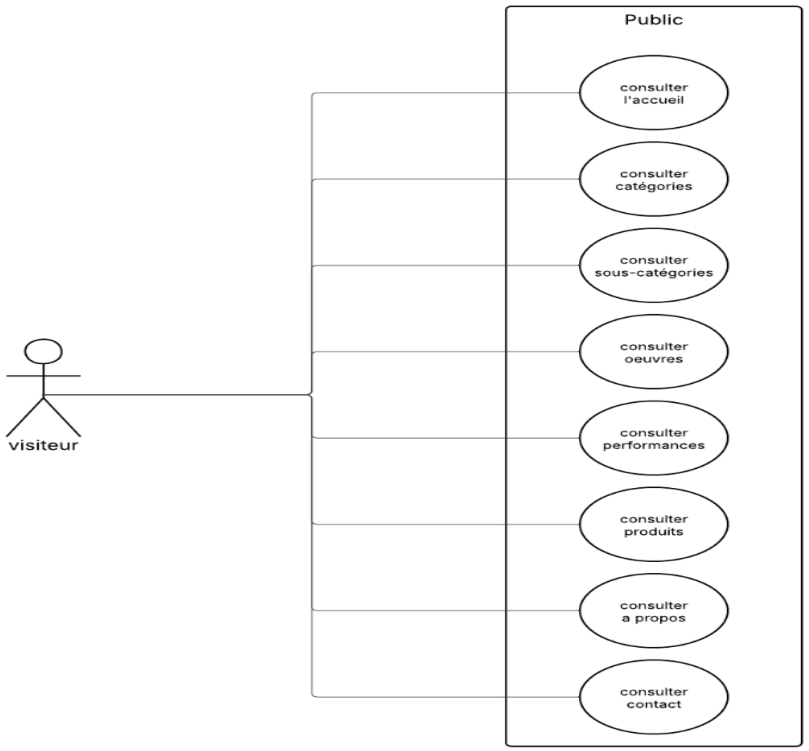
- Gestion des œuvres :
    - Ajouter des tableaux dans l'espace administrateur
    - Modifier ou supprimer les œuvres existantes
- Présentation des œuvres :
    - Page galerie pour exposer les tableaux
    - Organisation par catégories et sous-catégories
- Pages du site :
    - Page "À propos" présentant l'artiste
    - Page "Contact" (informations de contact)
    - Lien vers une galerie marchande externe pour l'achat
- Sécurité et administration :
    - Formulaire de connexion caché pour l'administrateur

2.2.3 Besoins non fonctionnels

- Protection des œuvres :
    - Empêcher le clic droit sur les images
    - Limiter l'accès aux images pour les IA (balises, headers)
- Accessibilité et compatibilité :
    - Interface responsive : compatible Desktop, Tablette et Téléphone
- Contraintes de projet :
    - Pas de délais imposés
    - Analyse des options d'hébergement et recherche de nom de domaine

2.2.4 Diagramme de cas d’utilisation





2.3 Maquettage

Outil utilisé

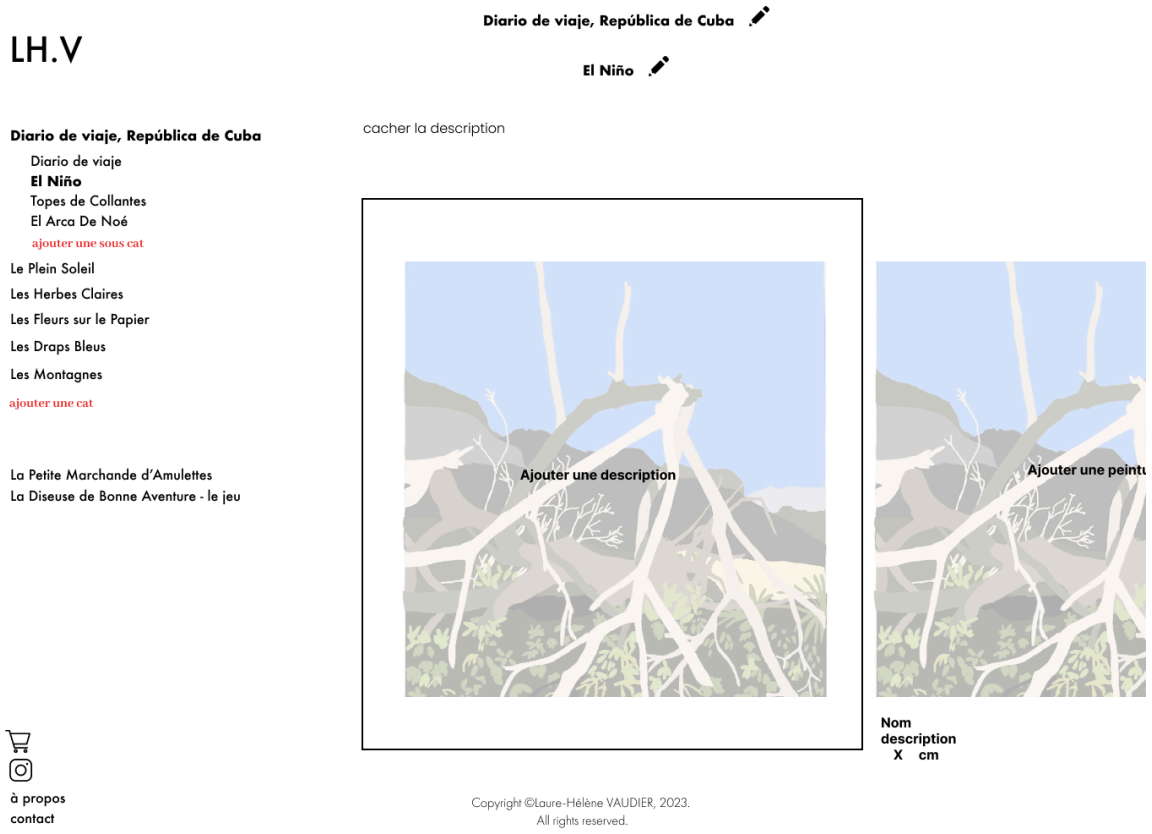
Les maquettes ont été réalisées à l'aide de **Figma**. Le client avait fourni un Figma initial avec toutes les pages prévues et ses idées pour la structure du site.

Maquettes écran par écran

- **Pages existantes fournies par le client** :
  - Page d'accueil
  - Galerie / exposition des œuvres
  - Page "À propos"
  - Page contact
- **Apport de l'équipe** :
  - Proposition d'un **panel administrateur reprenant la même interface que la vue publique** afin d'assurer une expérience cohérente pour l'administrateur.
  - Création complète de la maquette du panel admin sur Figma, intégrant l'ajout, la modification et la suppression des œuvres.

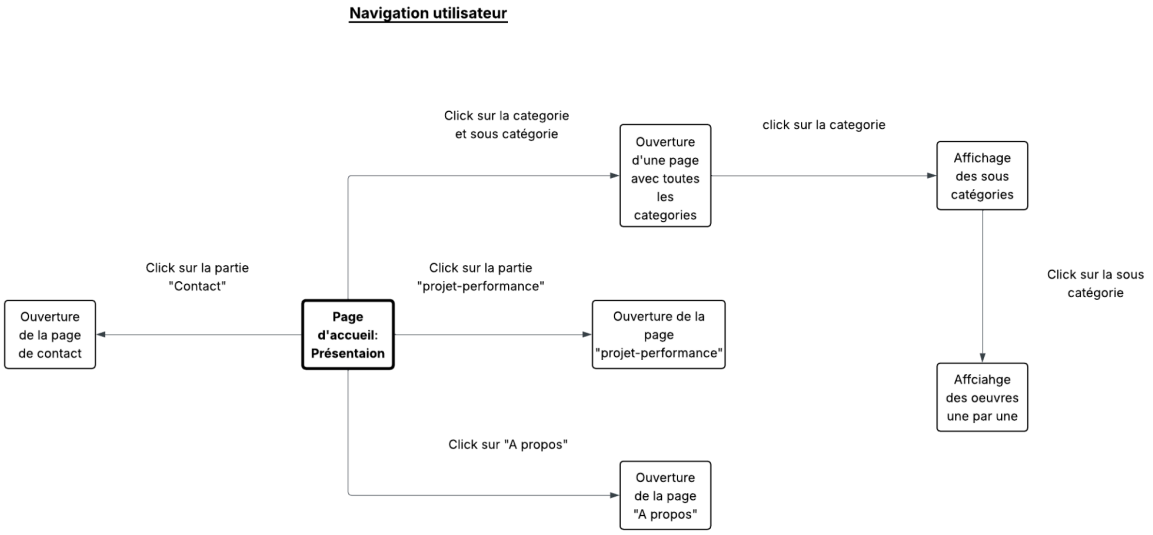
exemple de maquette partie public :

exemple de maquette partie d’administrateur :

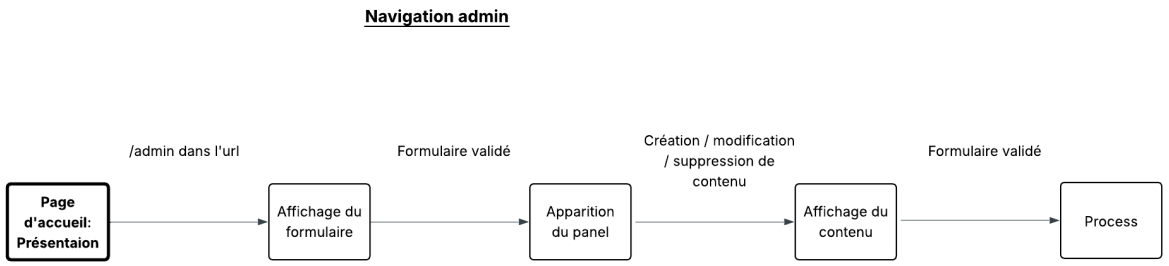


2.4 Parcours Utilisateur

Visiteur



Admin



2.5 Dossier de conception

2.5.1 Architecture fonctionnelle

L'application est organisée autour de deux périmètres fonctionnels principaux :

- La consultation du site vitrine accessible à tout visiteur
- La gestion du catalogue réservée à un administrateur authentifié

Périmètre public (site vitrine)

Le visiteur peut :

- Consulter la page d'accueil
- Naviguer dans les catégories et sous-catégories
- Afficher les œuvres (paints), performances, produits associés et expositions
- Accéder à la page « À propos »

Aucune authentification n'est requise pour ces fonctionnalités.

Périmètre administration

L'administrateur accède à une page de connexion dédiée.

Une fois authentifié, il peut :

- Créer, modifier, supprimer et réorganiser :
    - Catégories

    - Sous-catégories
    - Œuvres
    - Performances
    - Produits
    - Expositions

- Se déconnecter pour clôturer la session

L'espace administrateur est protégé et accessible uniquement aux utilisateurs disposant du rôle adéquat.

L'architecture fonctionnelle repose sur trois blocs :

1. Consultation du catalogue (site vitrine)
2. Authentification et autorisation (connexion, rôles, protection des routes)
3. Gestion du catalogue (CRUD et gestion des médias)

Les blocs consultation et authentification interagissent avec le bloc gestion afin de permettre l'administration sécurisée des contenus.

2.5.2 Contraintes techniques

Les contraintes suivantes ont été prises en compte lors de la conception.

Hébergement et exploitation

- Hébergement compatible avec une architecture front-end, API et base de données
- Utilisation d'une base de données managée : PostgreSQL via Supabase
- Stockage dédié pour les images (originaux et versions optimisées)

Performance

- Temps de réponse optimisé pour la consultation
- Optimisation des images (poids et formats adaptés)

- Réduction de la bande passante pour améliorer l'expérience utilisateur

Sécurité

- Authentification obligatoire pour l'accès à l'administration
- Contrôle d'accès par rôle (administrateur)
- Validation des entrées côté API
- Utilisation de requêtes paramétrées pour limiter les risques d'injection

Compatibilité

- Support des navigateurs modernes compatibles JavaScript et API REST
- Interface responsive adaptée aux différents types d'appareils

Contraintes projet

- Stack technologique maîtrisée par l'équipe : Go, React / Next.js, PostgreSQL
- Réutilisation de services managés (authentification, base de données, stockage)

2.5.3 Choix technologiques

Les choix technologiques répondent aux objectifs fonctionnels et aux contraintes identifiées.

Front-end

Framework : Next.js (React)

- Framework adapté à une application web moderne
- Routing intégré
- Possibilité de rendu côté serveur
- Bonne maintenabilité et évolutivité

Langage : TypeScript

- Typage statique

- Réduction des erreurs
- Amélioration de la lisibilité et de la maintenabilité

Appels API : Axios + React Query

- Axios pour centraliser les appels HTTP et gérer les intercepteurs (token, erreurs)
- React Query pour la gestion du cache et des états de chargement

Authentification (front) : Supabase Client

- Gestion des sessions (login, logout)
- Récupération du JWT
- Cohérence avec la vérification côté back-end

Back-end

Langage et framework : Go + Gin

- Performances élevées
- API REST claire et maintenable
- Déploiement simple

Accès aux données : GORM

- ORM pour PostgreSQL
- Requêtes paramétrées pour la sécurité
- Structs typées
- Réduction du SQL manuel

Authentification (back) : JWT Supabase (JWKS)

- Vérification des tokens émis par Supabase
- Pas de gestion directe des mots de passe dans l’API

- Sécurisation des routes administrateur

Données et stockage

Base de données : PostgreSQL (Supabase)

- Base relationnelle adaptée au modèle de données (catégories, œuvres, produits, expositions, profils)
- Offre managée limitant l'administration serveur

Stockage des fichiers : Supabase Storage

- Gestion des images (originaux et versions optimisées)
- Buckets sécurisés
- Intégration cohérente avec l'authentification

Architecture globale

Architecture 3 tiers avec API REST :

1. Présentation : Front-end
2. Logique métier : API Go
3. Données : PostgreSQL et stockage

Communication en JSON via API REST permettant une séparation claire des responsabilités, une meilleure testabilité et une évolution possible vers d'autres clients (mobile ou autre front).

3. Définir l'architecture logicielle

3.1 Choix de l'architecture

Architecture globale

Le projet repose sur une architecture 3 tiers composée de :

Front-end (présentation)
Application web développée avec Next.js (React), responsable de l'interface utilisateur, de la navigation et des interactions.

Back-end (logique métier / API)
API HTTP développée en Go avec le framework Gin, exposant des endpoints REST et appliquant les règles de gestion.

Base de données (données)
Base relationnelle PostgreSQL via Supabase, utilisée pour persister les entités métier (catégories, œuvres, performances, produits, expositions, profils, etc.).

Cette séparation permet :

- Une maintenabilité accrue grâce à une répartition claire des responsabilités
- Une évolutivité facilitée (le front peut évoluer indépendamment du back-end)
- Une meilleure sécurité (contrôle d'accès côté API et protection des routes côté front)
- La réutilisation possible de l'API par d'autres clients (application mobile, autre front-end)

3.2 Architecture multicouche

Le back-end suit une architecture multicouche.

Couche Présentation

- Front-end Next.js / React.
- Responsable de l'interface utilisateur et de la consommation de l'API REST.

Couche Métier
Services back-end implémentant les règles de gestion :

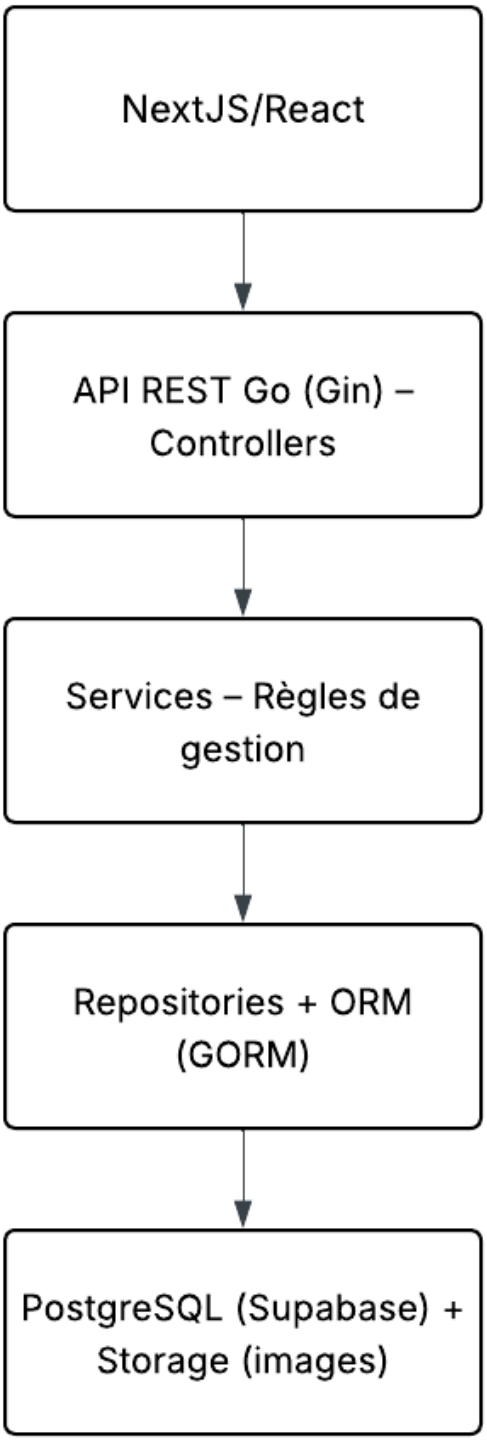
- Calcul d'index
- Gestion des transactions
- Cohérence entre entités et fichiers associés

Couche Accès aux données

- Repositories + ORM (GORM).
- Gestion du CRUD, pagination, filtres et tri.

Base de données et stockage

- PostgreSQL (via Supabase) et Supabase Storage pour les images.



3.3 Rôle des couches

Couche Présentation

- Affiche les données (catalogue public, espace administrateur)
- Déclenche les actions utilisateur
- Appelle l'API REST
- Gère les états (chargement, erreurs)

Couche Métier

Implémente les règles propres au domaine :

- Calcul automatique d'un ordre d'affichage
- Transactions pour garantir la cohérence
- Suppression cohérente des entités et de leurs images

Couche Data

- Centralise l'accès à la base
- Gère les opérations CRUD
- Implémente pagination, filtres et tri
- Protège contre l'injection SQL via des requêtes paramétrées (ORM)

3.4 Sécurité

Authentification

Le projet utilise le système d'authentification de Supabase :

- Le front réalise la connexion et obtient un JWT
- Le back-end vérifie le token sur les routes protégées

Autorisation

L’accès à l’administration est contrôlé par un rôle (ex. admin).

- Certaines routes REST sont accessibles uniquement si le token est valide et que le rôle requis est présent
- Le front protège également les routes /admin côté interface

Gestion des mots de passe

Le hashage et la gestion des mots de passe sont délégués à Supabase Auth (GoTrue).
L’API Go ne manipule pas directement les mots de passe.

Protection contre les vulnérabilités

Injection SQL
Utilisation de GORM et de requêtes paramétrées évitant la concaténation de requêtes SQL.

XSS

- Validation et contrôle des contenus affichés

CSRF

- Utilisation d’un token Bearer dans le header Authorization
- Pas de cookies de session classiques côté API
- Configuration CORS limitant les origines autorisées

3.5 Éco-conception

Optimisation des requêtes

- Structuration via repositories
- Pagination, filtres et tri
- Transactions uniquement lorsque nécessaire

Réduction des appels API

- Client API centralisé
- Mise en cache côté front avec React Query

Optimisation des images

- Séparation images originales / images optimisées
- Suppression des images associées lors de la suppression d'une entité

Choix d'infrastructure

- Services managés (base de données, authentification, stockage) limitant l'infrastructure à maintenir
- Possibilité de choisir une région d'hébergement proche du public cible afin de réduire la latence et l'impact énergétique

4. Concevoir et mettre en place une base de données relationnelle

4.1 Analyse des données

L'analyse des besoins fonctionnels a permis d'identifier les principales entités métier et leurs relations pour le site vitrine et l'espace administrateur.

Entités et rôle fonctionnel

- **User** (name, email, password) : représente les utilisateurs du système.
- **Role** (name) : définit les droits d'accès des utilisateurs.
- **Category** (name, index) : regroupe les œuvres par thématique.
- **Sub_cat** (name, index) : subdivision d'une catégorie.
- **Paint** (name, description, size, index, image) : œuvre individuelle du site.
- **Performance** (name, description, image) : projet artistique regroupant plusieurs produits.
- **Product** (name, description, image) : élément associé à une performance.
- **Expositions** (display_date, description, display) : événement affiché publiquement.

Relations et cardinalités

- **User – Role** : un utilisateur possède un seul rôle, un rôle peut être attribué à plusieurs utilisateurs.
- **Category – Sub_cat** : une catégorie contient plusieurs sous-catégories, chaque sous-catégorie appartient à une seule catégorie.
- **Category/Sub_cat – Paint** : une œuvre appartient obligatoirement à une catégorie et éventuellement à une sous-catégorie ; une catégorie ou sous-catégorie peut regrouper plusieurs œuvres.
- **Performance – Product** : une performance regroupe plusieurs produits, un produit appartient à une seule performance.
- **Expositions** : entité indépendante, sans relation avec les autres tables.

Lien avec les besoins fonctionnels

Cette analyse permet de structurer la base pour gérer :

- les utilisateurs et leurs rôles
- l'organisation et l'affichage des œuvres par catégorie et sous-catégorie
- l'association des produits aux performances
- la gestion des expositions sur le site vitrine

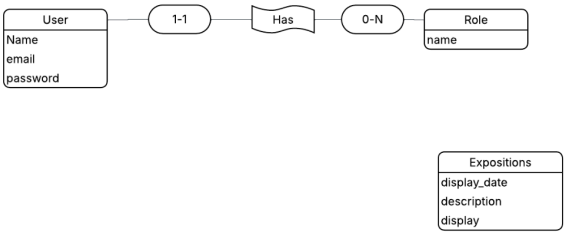
4.2 Modèle Conceptuel de Données (MCD)

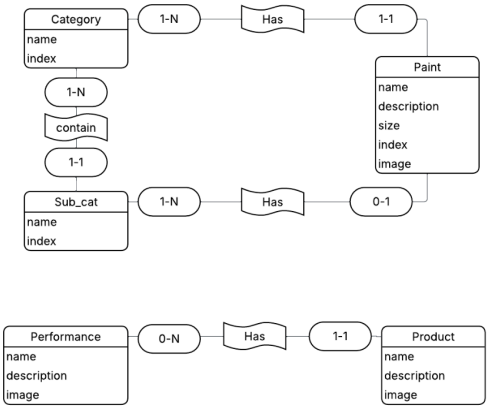
Le MCD formalise les entités et leurs relations identifiées lors de l'analyse, de manière indépendante de toute contrainte technique. Il sert de base pour créer le modèle logique (MLD) et le modèle physique (MPD).

Points principaux

- Huit entités : User, Role, Category, Sub_cat, Paint, Performance, Product, Expositions
- Relations et cardinalités indiquées (1-n, facultatif/obligatoire pour Paint → Sub_cat)

- Identifiants uniques pour chaque entité
- Représentation graphique sous forme de diagramme MERISE ou UML





4.3 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en **tables relationnelles exploitables par un SGBD**, avec des clés primaires et étrangères qui matérialisent les relations.

Tables et relations principales

- **Expositions** (id, display_date, description, display) : table indépendante.
- **Category** (id, name, index) et **Sub_cat** (id, name, index, cat_id) : liées par `cat_id`.
- **Paint** (id, name, description, size, index, image, cat_id, sub_cat_id) : référence Category et Sub_cat.
- **User** (id, name, email, password, role_id) et **Role** (id, name) : liées par `role_id`.
- **Performance** (id, name, description, image) et **Product** (id, name, description, image, perf_id) : liées par `perf_id`.

Clés et intégrité

- **Clés primaires** : identifiants uniques (UUID)

- **Clés étrangères** : matérialisent les relations du MCD et assurent l'intégrité référentielle

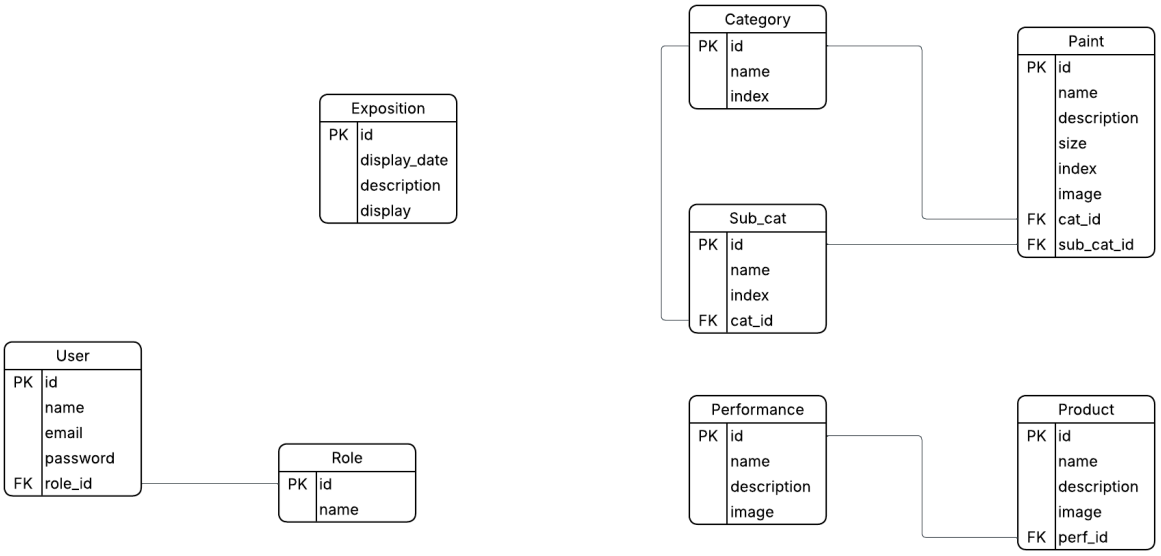
- **Index** : peuvent être ajoutés sur les clés étrangères pour optimiser les requêtes et les jointures

Gestion des utilisateurs

- L'authentification est gérée via **Supabase Auth**.

- Chaque utilisateur possède un **id interne** (clé primaire UUID) et un **auth_id** fourni par Supabase.

- Le auth_id permet de relier les données métier (œuvres, performances) au compte authentifié.



4.4 Modèle Physique de Données (MPD)

Le modèle physique décrit l'implémentation réelle des tables dans le **SGBD PostgreSQL**. Il précise les types de données, les clés primaires et étrangères, et les index utilisés.

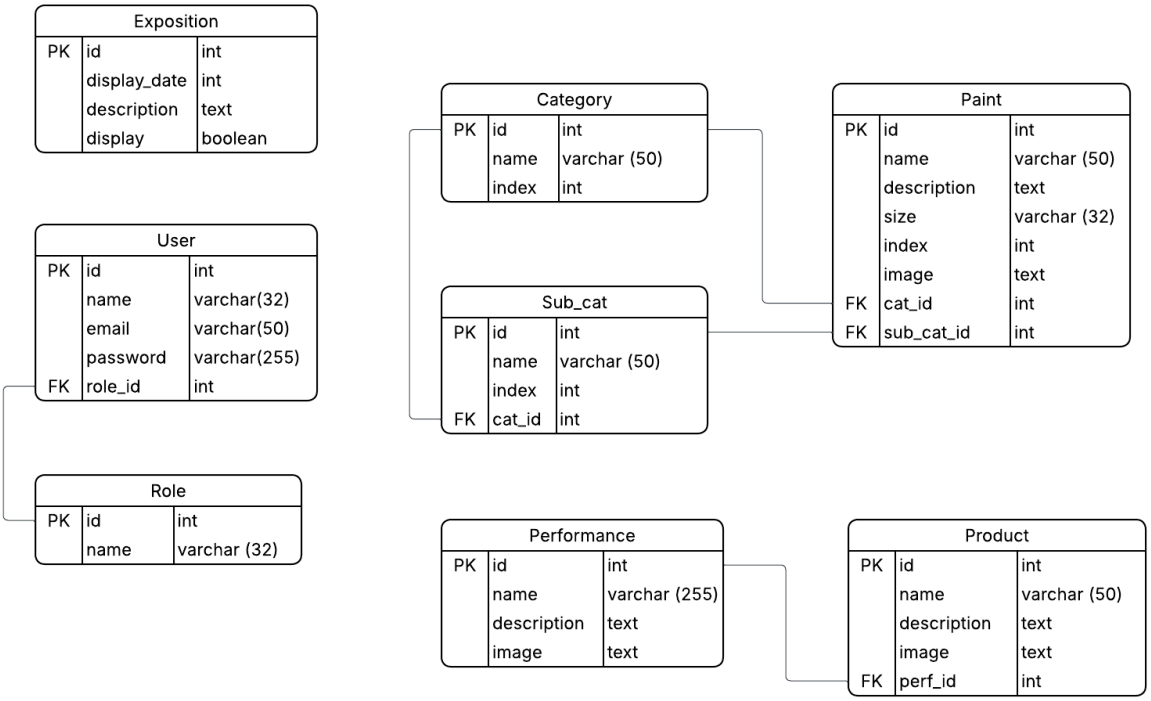
À titre d'illustration, voici la création de la table categories telle qu'elle est générée dans le projet :

```sql
CREATE TABLE categories (
 id BIGSERIAL PRIMARY KEY,
 created_at TIMESTAMP WITH TIME ZONE,
 name VARCHAR(255),
 "index" SMALLINT,
 image VARCHAR(500),
 image_original VARCHAR(500)
)
```

Explications :

- id BIGSERIAL : identifiant numérique auto-incrémenté de la catégorie
- created_at TIMESTAMP WITH TIME ZONE : date de création de l'enregistrement
- name VARCHAR(255) : nom de la catégorie
- "index" SMALLINT : position ou ordre d'affichage
- image et image_original : chemins vers les images optimisées et originales

Les autres tables suivent une logique similaire, avec **id numériques auto-incrémentés** pour les entités classiques et **UUID** pour **profiles** afin de rester compatible avec **Supabase Auth**.



4.5 Règles de nommage

Les noms des tables et des colonnes respectent une convention commune afin d'assurer la cohérence et la lisibilité du schéma.

Les tables et colonnes sont écrites en snake_case, en minuscules avec des underscores. Cette convention garantit une homogénéité globale et évite les conflits avec les mots réservés SQL.

Les noms sont explicites et reflètent clairement leur rôle fonctionnel. Les clés étrangères reprennent le nom de la table cible suivi de _id, ce qui permet d'identifier immédiatement les relations entre les entités.

La majorité des tables utilisent une clé primaire nommée id, auto-incrémentée. La table des rôles utilise le champ name comme clé primaire, car il représente un identifiant métier unique.

La colonne index, utilisée pour l'ordre d'affichage, correspond à un mot réservé SQL et nécessite une gestion particulière dans les requêtes.

4.6 Intégrité et sécurité

L’intégrité des données est assurée par la mise en place de contraintes adaptées au modèle relationnel.

Les champs indispensables au fonctionnement métier sont définis en NOT NULL, notamment les identifiants, les libellés obligatoires et les clés étrangères requises. Les champs optionnels restent nullables lorsque la règle métier le permet.

Des contraintes UNIQUE sont appliquées lorsque l’unicité est nécessaire, comme pour le nom des rôles ou les identifiants d’authentification gérés par Supabase. D’autres colonnes peuvent être rendues uniques selon les besoins métier afin d’éviter les doublons.

Les relations entre les tables sont garanties par des clés étrangères assurant la cohérence référentielle. Les règles de mise à jour et de suppression sont définies en fonction des contraintes métier afin d’éviter toute suppression incohérente ou perte de données liée.

Les opérations impliquant plusieurs actions en base sont exécutées dans des transactions afin de garantir l’atomicité. En cas d’erreur, un mécanisme de rollback permet de revenir à un état cohérent.

Enfin, la sécurité d’accès à la base est gérée par le SGBD et la plateforme Supabase. L’application dispose uniquement des droits nécessaires aux opérations courantes, sans privilèges d’administration en production, ce qui limite les risques de manipulation accidentelle ou malveillante de la structure des données.

4.7 Jeu d’essai

Un jeu d’essai est prévu pour le développement et les tests afin de disposer de données réalistes et reproductibles. Les données sont insérées automatiquement au démarrage de l’application en mode debug ou via un script SQL exécutable manuellement, permettant de restaurer le jeu d’essai après un reset de la base.

Les valeurs utilisées reflètent le domaine métier : noms de catégories et sous-catégories, œuvres, performances et expositions avec dates et descriptions. Les index d’ordre sont renseignés pour tester l’affichage et le tri.

Le jeu couvre les cas normaux, comme plusieurs sous-catégories par catégorie, œuvres rattachées à une catégorie, produits liés à une performance et expositions affichées, permettant de valider les parcours utilisateurs et les opérations CRUD.

Il inclut également des cas limites, comme des œuvres ou expositions sans sous-catégorie, des champs optionnels à NULL, des valeurs extrêmes pour les index ou display à false. Ces cas assurent que l’application et les requêtes se comportent correctement en toutes situations.

4.8 Documentation technique

Description des tables

- **categories** : regroupe les thèmes du catalogue. Chaque enregistrement contient un nom, un ordre d'affichage et des chemins vers les images optimisées et originales.

- **sub_categ** : sous-catégories liées à une catégorie via cat_id. Contient également nom, index et images.

- **paints** : œuvres (tableaux, etc.), rattachées à une catégorie et éventuellement à une sous-catégorie. Inclut nom, description, taille, index, orientation (landscape), taille d'affichage et chemins d'images.

- **performances** : spectacles ou projets artistiques. Contient nom, description et images. Pas de clé étrangère.

- **products** : produits liés à une performance via perf_id. Contient nom, description, index et images.

- **expositions** : événements affichés sur le site. Champs : année/date d'affichage, description et indicateur d'affichage (display). Table indépendante.

- **profiles** : utilisateurs avec identifiant UUID, nom, email et rôle (roleName), complétant les données Supabase Auth.

- **roles** : rôles applicatifs (admin, owner, user). La clé primaire est le nom du rôle.

Relations principales

- Une sous-catégorie appartient à une catégorie : sub_categ.cat_id → categories.id

- Une œuvre appartient à une catégorie et éventuellement à une sous-catégorie : paints.cat_id → categories.id, paints.sub_cat_id → sub_categ.id

- Un produit appartient à une performance : products.perf_id → performances.id

- Un profil a un rôle : profiles.roleName → roles.name

- Aucune table ne référence expositions.

Dictionnaire de données (résumé)

Chaque table possède une clé primaire unique, des dates de création et des champs métiers clairs. Les clés étrangères respectent les relations décrites, et les champs optionnels sont nullables si nécessaire. Les champs image et image_original contiennent les chemins vers les fichiers dans le bucket de stockage. Les indicateurs tels que index ou display permettent de gérer l'ordre et la visibilité dans l'application.

Cette documentation assure la compréhension du modèle, la maintenabilité de la base et facilite les tests et le développement des composants applicatifs.

5. Développer des composants d'accès aux données SQL et NoSQL

5.1 Technologies utilisées

- **Base SQL** : PostgreSQL (Supabase, local ou cloud). Gestion des tables, clés étrangères et transactions.
- **NoSQL** : non utilisé.
- **ORM** : GORM (Go). Il traduit les structures de données Go (structs) en tables PostgreSQL, gère les relations entre tables, les requêtes sécurisées, les transactions et les migrations de la base.

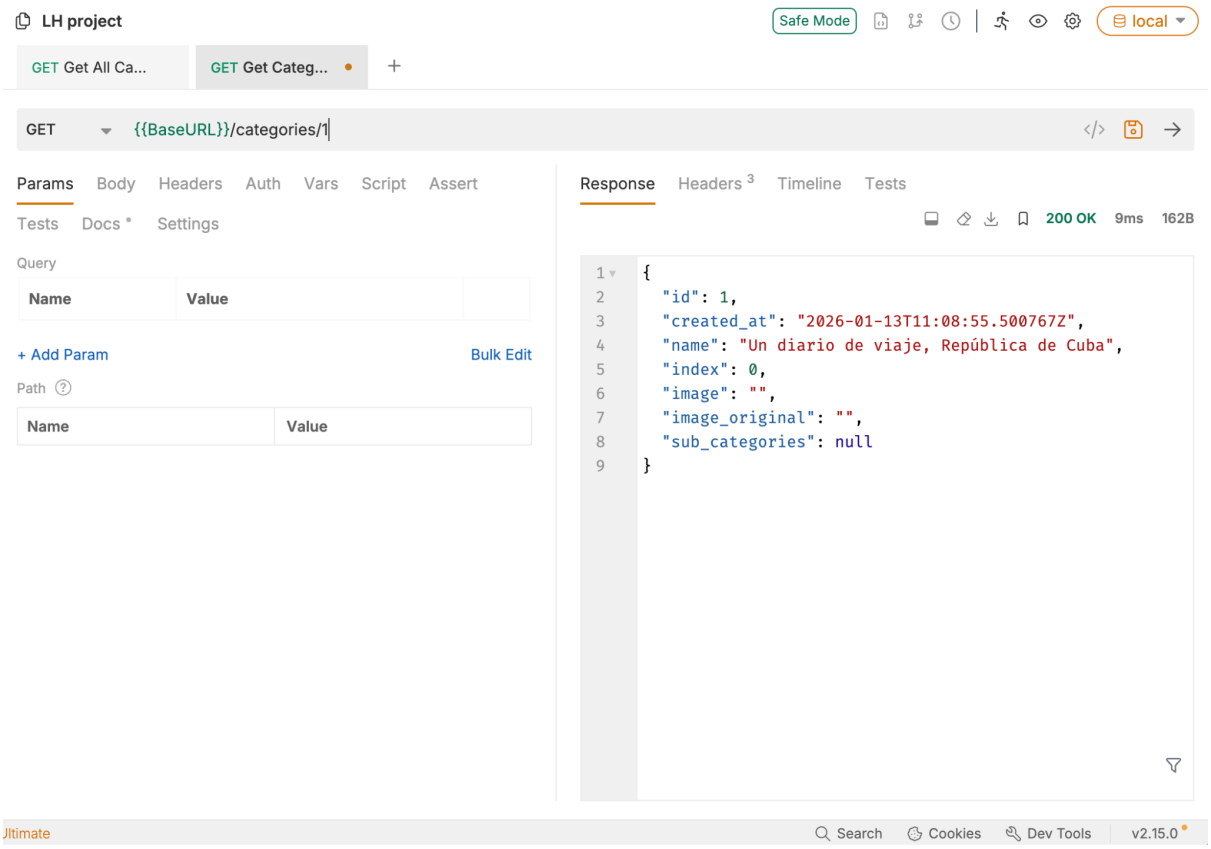
5.2 Développement des composants

Architecture en couches : **Repository** → **Service** → **Controller**

- **Repository** : interface CRUD générique + méthodes métier spécifiques (pagination, filtres, tri).
- **Service** : orchestre repository et règles métier (calcul d'index, suppression d'images, rollback sur erreur).
- **Controller** : reçoit requêtes HTTP, appelle service, renvoie JSON avec codes HTTP adaptés (200, 201, 400, 404, 500).

Exemple de flux :

GET /categories/:id → Controller GetCategoryByID → Service GetByID → Repository FindByID → GORM → PostgreSQL → réponse JSON



5.3 Gestion des exceptions

- Controller : 400 pour ID invalide, 404 si enregistrement non trouvé, 500 pour autres erreurs.
- Service : rollback transaction en cas d'erreur.
- Repository : GORM renvoie les erreurs à traiter en amont.

5.4 Validation des entrées

- URL : ID parsé, 400 si invalide.
- JSON : binding avec Gin, 400 si invalide.
- Upload fichiers : vérification MIME, taille, table autorisée.

5.5 Sécurité et middlewares

Les opérations sensibles (création, modification, suppression) sont protégées par des **middlewares** exécutés **avant** le controller. Ils garantissent que seuls les utilisateurs authentifiés et autorisés accèdent à ces routes.

- **Initialisation JWKS** : *InitJWKS()* charge les clés publiques Supabase pour vérifier les JWT et les rafraîchit périodiquement pour gérer la rotation des clés.

- **RequireAuth()** : lit le token JWT dans l'en-tête Authorization, le valide via JWKS, puis stocke les informations de l'utilisateur (claims et identifiant) dans le contexte Gin. En cas de token absent ou invalide, renvoie **401 Unauthorized**.

- **RequireRole(allowedRoles)** : récupère le rôle de l'utilisateur depuis les claims et vérifie qu'il est autorisé. Sinon, renvoie **403 Forbidden**.

Toutes les opérations sensibles ne sont donc exécutées qu'après vérification du token et du rôle.

5.6 Tests

- Unitaires et intégration : testing + testify, base de test initialisée pour chaque package.

- Sécurité : tests middlewares auth et rôle.

- Exécution : go test ./... pour tests, go test -cover ./pkg/... pour couverture.

```
guangquanye@pc-86 back % go test ./pkg/...
?       back/pkg/config [no test files]
?       back/pkg/db     [no test files]
?       back/pkg/handlers       [no test files]
ok      back/pkg/handlers/categories    (cached)
ok      back/pkg/handlers/expositions   (cached)
ok      back/pkg/handlers/paints        (cached)
ok      back/pkg/handlers/performances  (cached)
ok      back/pkg/handlers/products      (cached)
ok      back/pkg/handlers/profiles      (cached)
ok      back/pkg/handlers/subcateg      (cached)
ok      back/pkg/handlers/upload        (cached)
ok      back/pkg/middleware     (cached)
```

Conclusion

Ce projet a permis de concevoir et de développer un site vitrine artistique structuré autour d'une galerie organisée par catégories et sous-catégories, ainsi qu'un espace d'administration sécurisé reposant sur l'authentification et la gestion des rôles.

L'architecture en trois tiers (front-end Next.js, API Go, base PostgreSQL via Supabase) et le découpage en couches (repository, service, controller, middlewares) ont assuré une organisation claire du code, la cohérence des données et la sécurisation des accès. La base de données a été modélisée et documentée (MCD, MLD, MPD) afin de garantir sa maintenabilité et son évolutivité.

Les objectifs initiaux — centraliser les œuvres, permettre une gestion autonome du catalogue et préparer une évolution vers une solution e-commerce intégrée — sont atteints. Les évolutions futures porteront principalement sur la mise en place complète de la vente en ligne et le déploiement en production.